XJTLU | Computing

CPT111: Java Programming

Semester 1, 2024-25

Laboratory 6: Methods - Parasite simulation

1 Purpose

In this lab, you are going to simulate a forest plagues by a parasite.

A parasite plagues a forest by infecting the trees and causing them to defoliate (lose their leaves). Defoliated trees are removed from the forest and new trees are planted in their places, which in turn may become infected by the parasite. The following three rules define a simple model of the forest environment.

1. An ***infected area*** becomes ***defoliated*** the following year;
2. A ***defoliated*** area becomes *green* the following year;
3. An infected area ***infects*** its neighbours to the north, south, east and west the following year, if they are currently *green*.

A biologist requires a computer program that will use the rules given above to simulate the changes in the forest over a number of generations (one generation = one year). The program should represent the state of the forest by a grid of 25 letters. For example, the state

```
G G G G G
G G I G G
G G D D G
G G G G G
G G G G G
```

indicates that most of the forest is green (`G`) with only one infected area (`I`) and two defoliated areas (`D`). The user of the program should be able to specify the initial state of the forest, and to enter the number of generations for which the simulation will run.

2 Tasks

There are two parts for this lab.

Task 1

As can be found in the downloaded package, below is the skeleton code for the lab. Design and implement necessary functions for the following `main()` function to run properly:

```
public class ParasiteSimulation {
  public static void main(String... arguments) {
    // initial states of the forest
    char[][] forestState = ... ;

    // the number of generations to simulate
    int numOfGenerations = 30;

    // your code start here!

  }
}
```

To simplify the scope of the lab exercise, you can assume the initial state of the forest is stored in a 2-D array, as indicated above.

This is **NOT** a hard problem, but it is easy to get a messy program if you are not careful. So don't rush into writing the codes, think about how to solve the problem in the cleanest possible way.

Before start implementing your code, you should start analysing the problem by identifying:

- Input: What is given?
- Output: What is required?
- Constraints: Under what conditions?
- Abstraction: What information are essential?

ALL must be presented in some form of data that can be processed by computer.

Task 2

One problem with the simulation program above is that it runs for a fixed number of generations regardless of whether the state of the forest is stable or not. For instance, if the initial state of the forest is all green, then it stays that way forever. Modify the above simulation program so that it stops as soon as the state becomes stable.

To see the difference between Tasks 1 and 2, try the following initial state, and simulate it for 30 generations.

```
G G I G D
G G G G G
G G G G G
G G G G G
G G G G G
```

3 Important reminders

Documentation is also a critically important part of your software engineering. Your use of comment (in Java source file) will be graded.

You must write the final version of the program ***totally on your own***. Sophisticated plagiarism detection system are in operation, and they are pretty good at catching copying! Re-read the policy on the university home page, and note the University's tougher policy this year regarding cheating.

Using libraries that are **NOT** covered in this course *or* the Java Collection package, i.e., classes in `java.util.*` package, is ***strictly prohibited*** and will result in **zero (0)** marks automatically in this lab exercise.

Your programming style (how clearly and how well you speak a programming language, i.e., Java in this course) is what will be graded. Correct functioning of your program is *necessary* but ***not*** *sufficient*!

4 Grading scheme

Be prepared to demonstrate and explain your working program for your teaching assistants (TAs). The TAs will check your source code to make sure you made reasonable use of arrays and functions.

Your grade consists of two parts: (i) *program correctness*, worth 80%, and (ii) *style & design*, worth 20%. Points are deducted for the error types as shown (but the minimum you can get for each part is 0%, so you cannot get negative point scores on either program correctness or style & design).

#	Description	Points	Notes
Program Correctness (Maximum: 80%, Minimum: 0%)			
1	Test results for forest state after update (Task 1)	50	
2	Test results for stop updating the state when no further changes has been detected (Task 2)	30	
☹	Program prints irrelevant information	-10	
Style & Design (Maximum: 20%, Minimum: 0%)			
3	Base point	20	
☹	Incorrect usage of constant variable	-10	
☹	Poor readability of function and variable names	-10	
☹	No comment has been written	-20	
☹	Inadequate function level documentation	-10	
	Total	100	

You should *submit* your Java source files (extension `.java`) to Learning Mall (LM). If for some reason you cannot finish before the end of the lab period, upload it to LM by 23:59pm 27 October 2024.

Expected output from the first 10 updates of Task 1

```
Parasite Simulation

=============
Initial state
-------------
G G G G G
G G I G G
G G D D G
G G G G G
G G G G G
====================
Simulation state #1
-------------------
G G I G G
G I D I G
G G G G G
G G G G G
G G G G G
====================
Simulation state #2
-------------------
G I D I G
I D G D I
G I G I G
G G G G G
G G G G G
====================
Simulation state #3
-------------------
I D G D I
D G G G D
I D I D I
G I G I G
G G G G G
====================
Simulation state #4
-------------------
D G G G D
G G I G G
D G D G D
I D I D I
G I G I G
====================
Simulation state #5
-------------------
G G I G G
G I D I G
G G G G G
D G D G D
I D I D I
```

```
==================
Simulation state #6
------------------
G I D I G
I D G D I
G I G I G
G G G G G
D G D G D
==================
Simulation state #7
------------------
I D G D I
D G G G D
I D I D I
G I G I G
G G G G G
==================
Simulation state #8
------------------
D G G G D
G G I G G
D G D G D
I D I D I
G I G I G
==================
Simulation state #9
------------------
G G I G G
G I D I G
G G G G G
D G D G D
I D I D I
==================
Simulation state #10
------------------
G I D I G
I D G D I
G I G I G
G G G G G
D G D G D
```